

DATA SCIENCE · PARTE 2



De clientes dormindo a oportunidades de venda

com HistGradientBoosting

Parte 2 — construída sobre o medalhão de Engenharia (Parte 1). 20 features de RFM, ritmo e CRM. Score para 2.816 clientes. Fila de 200 contatos com motivo em português. O retorno do vendedor vira dado de treino.

20

FEATURES

200

CONTATOS/SEM

4

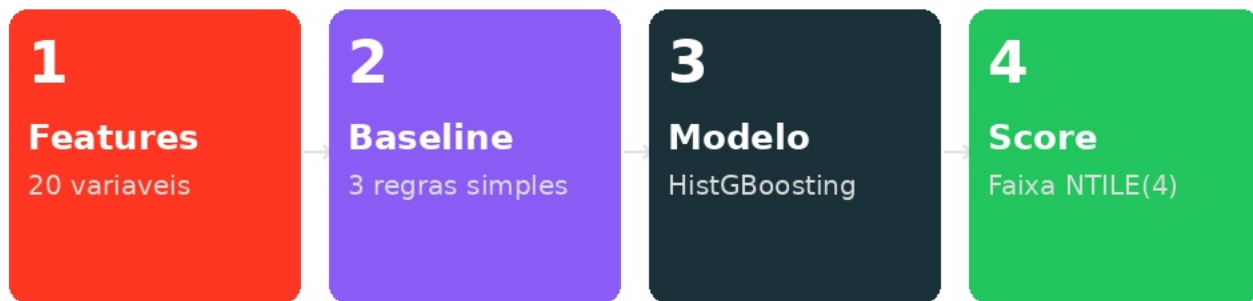
FAIXAS DE RISCO

Arraste para conhecer o pipeline de ponta a ponta

O PROBLEMA

Quem ligar primeiro?

Com 2.816 clientes ativos e sem critério de priorização, o time ligava por hábito — não por dado. A solução foi usar as tabelas Gold da Parte 1 como insumo para um score semanal de propensão de compra.



Por que HistGradientBoostingClassifier?

Trata NaN nativamente — sem imputação que poderia vaziar informação. Rápido, regularizado e robusto com datasets de porte médio. Alternativa ao XGBoost com menos dependência de ajuste fino.

3 asserts que quebram o job automaticamente:

`AUC >= baseline + 0.05`

`AUC < 0.99` (vazamento)

`lift_top200 >= 2.5x`

20 features, 4 grupos, zero vazamento

Função `montar_features(referência)` — todas as fontes filtradas por `data < referencia`. Nenhum dado futuro entra no treino.

RFM

6 features

- `recencia_dias`, `frequencia_pedidos`
- `valor_total`, `ticket_medio`
- `margem_total`, `margem_percentual`

RITMO

4 features

- `intervalo_medio_dias` (gap entre compras)
- `desvio_intervalo_dias` (regularidade)
- `atraso_relativo` = `recencia / intervalo` (teto 10)
- `pedidos_ultimos_90d`

CRM

5 features

- `oportunidades_abertas`, `oportunidades_g`
- `taxa_ganho` (conversao historica)
- `visitas_90d`, `conversao_visita`

MIX

5 features

- `skus_distintos`, `categorias_distintas`
- `marcas_distintas`, `concentracao_marca_top`
- `comprou_lancamento` (SKU lancado em 120d)

Baseline vs modelo: quanto ganhamos?

BASELINE (regras simples)

-recencia

ligar para quem
comprou
recentemente

valor_total

ligar para quem
compra mais

atraso_relativo

ligar para quem
esta atrasado

moeda

aleatorio
(referência)

HistGradientBoostingClassifier — resultado no holdout

AUC holdout

modelo > baseline
+ 0.05

lift top200

$\geq 2.5\times$ a taxa
base

acertos top200

dos 200 mais
cotados

3 asserts

job quebra se falhar

CALIBRAGEM DO MODELO (holdout)

Fria

taxa cresce
da Fria p/ Mq

Morna

taxa cresce
da Fria p/ Mq

Quente

taxa cresce
da Fria p/ Mq

Muito quente

taxa cresce
da Fria p/ Mq

IMPORTANCIA POR PERMUTACAO (top 5)

recencia_dias

atraso_relativo

intervalo_medio_dias

pedidos_ultimos_90d

valor_total

FILA SEMANAL

200 contatos priorizados, cada um com motivo em português e sugestao

O script 13-fila.sql monta a fila em 4 CTEs: elegibilidade (carteira vigente, vendedor ativo) > rank global por score > ROW_NUMBER por vendedor > SKU sugerido da marca preferida não comprado nos últimos 90 dias.

Exemplo de linha da fila_semanal:

vendedor:	Ana Paula
ordem:	1
razao_social:	Perfumaria Aroma Certo Ltda
score:	0.8734
faixa:	Muito quente
motivo:	Compra a cada 28 dias e esta ha 41 dias sem pedido. Risco de p
sugestao:	SKU-1042 (última marca preferida, não comprado em 90d)

4 SQL FUNCTIONS NO UNITY CATALOG:

`priorizar_carteira(vendedor, n)`

→ Top N da fila de um vendedor

`contexto_cliente(cliente_id)`

→ Histórico completo antes de ligar

`sugerir_produtos(cliente_id)`

→ SKUs parados na marca preferida

`checar_disponibilidade(sku)`

→ Saldo e ruptura no ultimo snapshot

3 testes: fila com 200 linhas | motivo não nulo | score em [0,1] — raise_error() se falhar

Databricks App: o vendedor age, o retorno vira dado

App React + TypeScript com Databricks AppKit UI, servido pelo Databricks Apps runtime. Três páginas, cada uma com uma responsabilidade clara.

A Semana

Fila dos 200 com filtro por vendedor. Cada linha exibe score, faixa, motivo e sugestão de SKU. O vendedor registra o retorno (Vendeu / Vai pensar / Sem interesse / Não atendeu) e um comentário livre.

Acompanhamento

Tabela por vendedor: total na fila, trabalhados e desfecho por categoria. Aparece vazia até o primeiro retorno ser registrado.

Perguntar

Genie Space 'Rota do Perfume - Direção' embutido no app. Pergunta em linguagem natural sobre fila, métricas e clientes.

Loop fechado:

Retorno do vendedor → retorno_ligacao → dado de treino do próximo modelo

MLflow, Unity Catalog e Genie com funções SQL

MLflow Tracking

Cada treino registra: algoritmo, hiperparâmetros, AUC, lift_top200, acertos_top200, baselines e feature_top (importância por permutação).

Unity Catalog Model Registry

Modelo registrado em lakehouse_rotaperfume.gold.propensão_compra. Alias @prod atualizado automaticamente após o treino. O scoring carrega pelo URI models:/...@prod.

Tabelas de metadados

modelo_metricas — uma linha por versão treinada. calibragem_holdout — taxa de compra por faixa no holdout. Ambas consultáveis pelo Genie Space.

Genie Space

Dois spaces no bundle: Comercial (vendedores) e Direção (gestores). Funções SQL do UC como certified functions — o Genie as reutiliza sem inventar SQL.

Disclaimer honesto:

O score indica propensão estatística de compra nos próximos 7 dias — não garante venda. Validado em holdout e monitorável via modelo_metricas. Retreinar quando o AUC cair ou a base de clientes mudar significativamente.

ENCERRAMENTO

O que esse projeto prova na prática

- Feature engineering com corte temporal evita vazamento — é o que separa um modelo útil de um ilusório.
- Baseline é obrigatório: sem ele não há como saber se o modelo vale o esforço.
- Loop fechado (retorno vira treino) transforma o modelo em produto vivo — não em experimento pontual.
- Asserts no job garantem que dado inconsistente não chega ao time comercial — qualidade como código.

STACK

PySpark · HistGradientBoostingClassifier · MLflow · Databricks UC
React + AppKit UI · Databricks Apps · Genie Space · SQL Functions

Código completo no GitHub

github.com/baludf/projeto_databricks

Parte 1 — Engenharia — disponível no mesmo repositório. Comente se quiser trocar ideia!

Alt: Encerramento — aprendizados, stack e link GitHub do projeto de Data Science.